



DOSW COMPANY

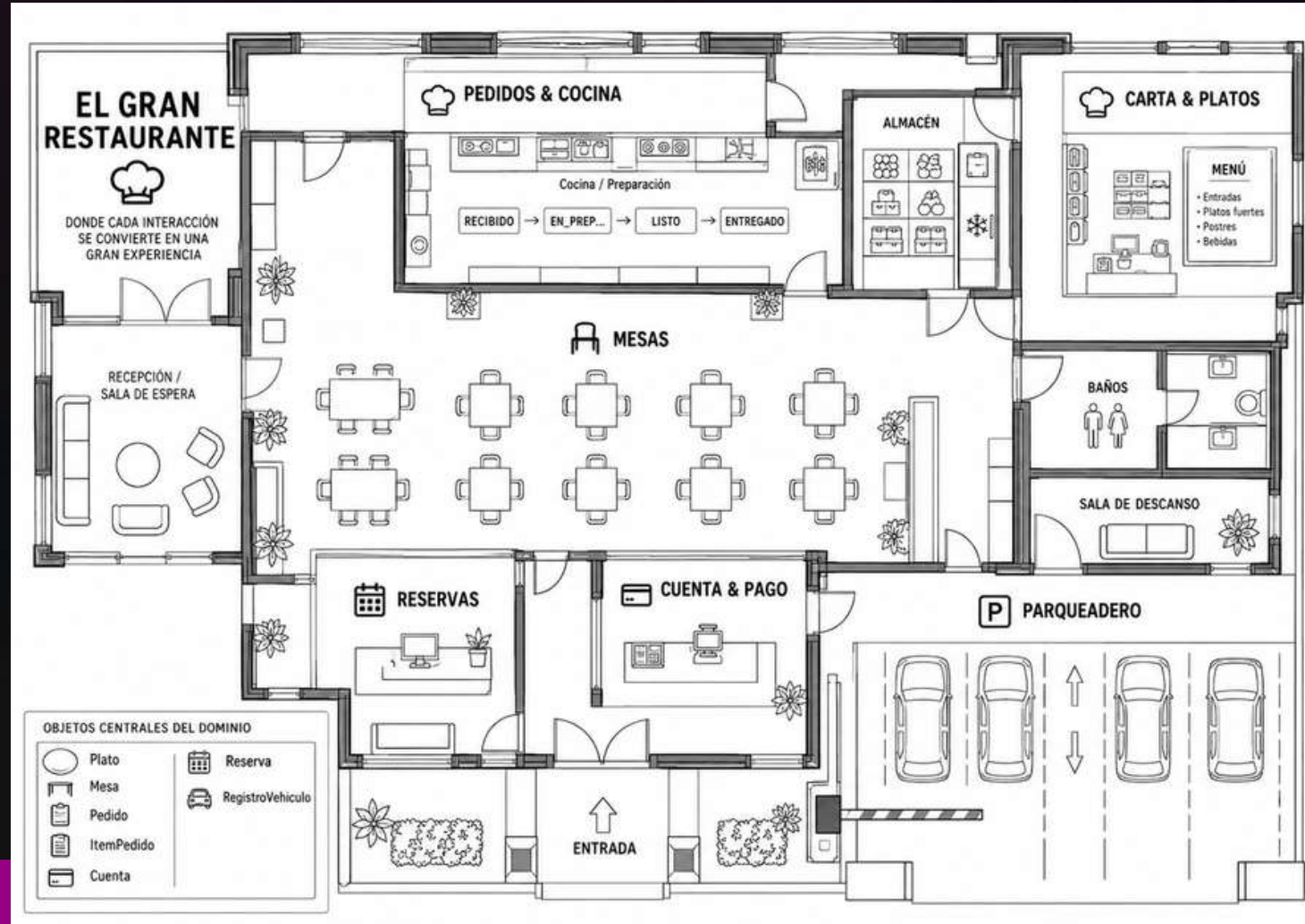


ESCUELA COLOMBIANA DE INGENIERIA

SEMANA 9

DOSW 1

EN QUE QUEDAMOS LA CLASE PASADA?





TU RESTAURANTE AHORA SI UTILIZA LA DESPENSA Y LA NEVERA

Ya construida nuestra aplicacion con los primeros
cimientos, es necesario que ahora si utilicemos la
persistencia correctamente

VAMOS A REVISAR COMO FUNCIONA SIN TENER DATOS EN MEMORIA

Funcionó para aprender la arquitectura. Esta
semana ese mapa se reemplaza con una base de
datos real. El resto de la arquitectura no cambia.



TODOS EN SUS COMPUTADORES



Van a ejecutar su aplicacion con spring y tendran su aplicacion corriendo en localhost:8080

Van a mostrar en su navegador su swagger corriendo, vamos a revisar si cumplieron con la asignacion para la clase de hoy

Ejecuten una funcionalidad de creacion, consulta y borrado para verificar que funcione



DOSW COMPANY

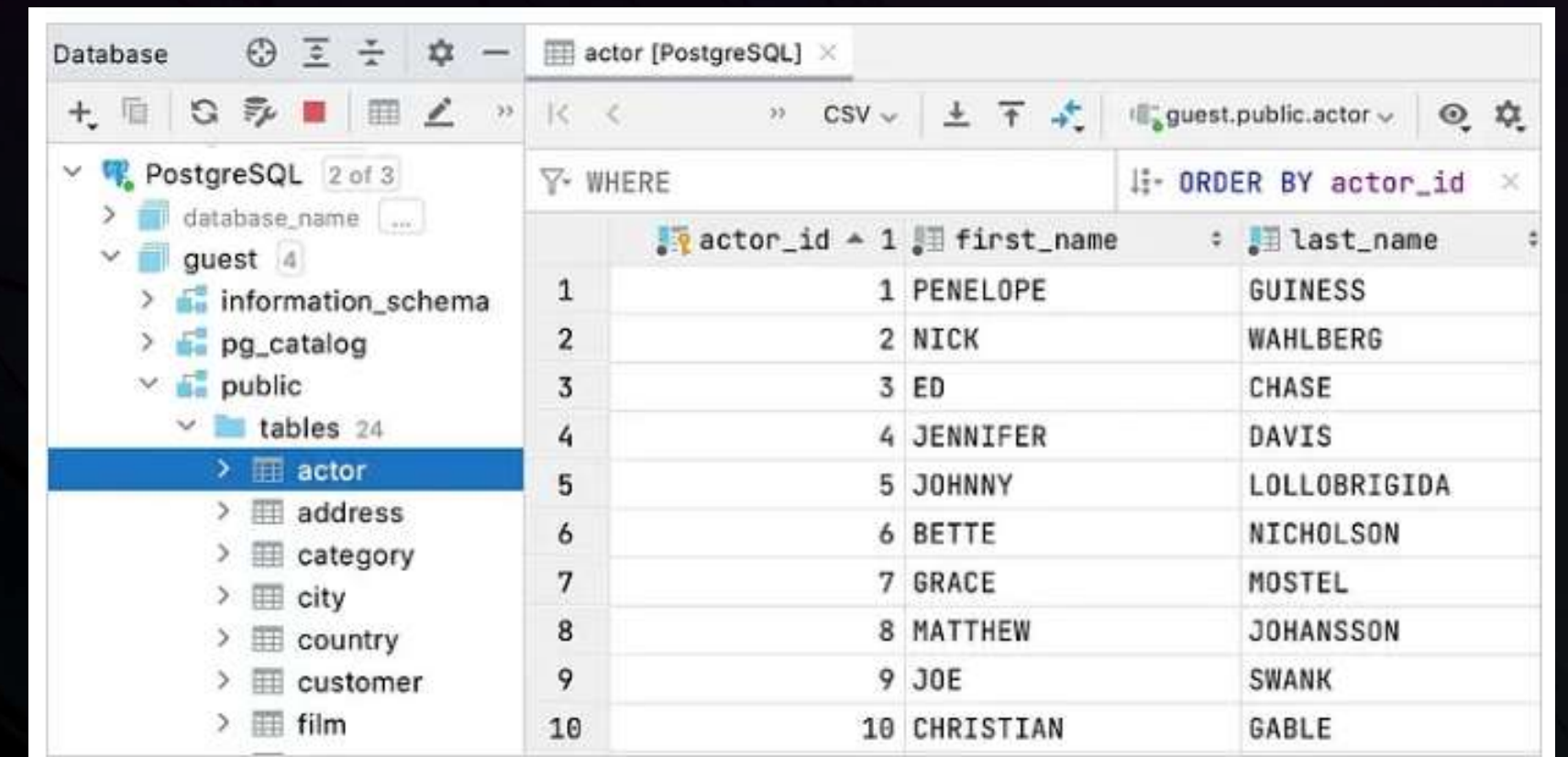
BASES DE DATOS RELACIONALES



BASE DE DATOS RELACIONAL - SQL

Es un tipo de base de datos que utiliza la logica de predicados y la teoria de conjuntos para organizar, relacionar y manipular la informacion

La idea es el uso de relaciones para el manejo y la organizacion de los datos, estableciendo relaciones logicas entre entidades permitiendo la consistencia y la integridad de la informacion



The screenshot shows a PostgreSQL database interface. On the left, a tree view displays the database structure: PostgreSQL (2 of 3) > database_name > guest (4) > information_schema > pg_catalog > public > tables (24) > actor. The 'actor' table is selected. On the right, the table's data is displayed in a grid view, ordered by actor_id. The table has three columns: actor_id, first_name, and last_name. The data is as follows:

actor_id	first_name	last_name
1	PENELOPE	GUINNESS
2	NICK	WAHLBERG
3	ED	CHASE
4	JENNIFER	DAVIS
5	JOHNNY	LOLLOBRIGIDA
6	BETTE	NICHOLSON
7	GRACE	MOSTEL
8	MATTHEW	JOHANSSON
9	JOE	SWANK
10	CHRISTIAN	GABLE

IMAGINEN...

Un cliente confundido porque le descontaron dinero de su cuenta pero no se ve el abono al destinatario, porque podría pasar esta situación?

Una tienda que mando un producto de su tienda pero no se realizo la actualizacion de su inventario, porque podría ser?

A.C.I.D

Se aplican para garantizar que cada transaccion se procese de forma fiable manteniendo la seguridad de los datos



A.C.I.D

Son cuatro propiedades que garantizan el procesamiento fiable de las transacciones de base de datos

ATOMICIDAD

Una transaccion se condisera una unica unidad e indivisible – Todas las operaciones dentro de una transaccion se completan en su totalidad o no, es decir si alguna parte falla el sistema se revierte por completo



CONSISTENCIA

Garantiza que una transaccion lleve la base de datos de un estado valido a otro, cumpliendo con las reglas o restricciones





A.C.I.D

AISLAMIENTO

Evita que las transacciones interfieran entre si, cuando se ejecutan simultaneamente, el aislamiento garantiza que no afecten los resultados de las demas

DURABILIDAD

Garantiza que una vez completada una transaccion sus cambios se almacenan permanentemente en BD, incluso si hay fallos



Detalles de una transacción en curso : 151115 (Referencia pedido: lw37395)			
Información Comprador Gestión de riesgos Análisis de riesgos Registro de actividad			
Fecha	Operación	Usuario	Inf.
05/05/2020 03:38:04	Llamada URL de notificación	...	SENT, rule=URL de notif...
05/05/2020 03:38:04	Llamada URL de notificación	...	REDIRECTED_301, rule...
05/05/2020 03:38:02	Modificación	...	84.6 ARS → 74.6 ARS
05/05/2020 03:37:48	Llamada URL de notificación	...	SENT, rule=URL de notif...
05/05/2020 03:37:48	Llamada URL de notificación	...	REDIRECTED_301, rule...
05/05/2020 03:37:47	Modificación	...	05/05/2020 → 07/05/2...
05/05/2020 03:37:22	E-mail de confirmación vende...	...	to: ...
05/05/2020 03:37:22	E-mail de confirmación compr...	...	to: ...
05/05/2020 03:37:22	Llamada URL de notificación	...	SENT, rule=URL de notif...
05/05/2020 03:37:22	Llamada URL de notificación	...	REDIRECTED_301, rule...
05/05/2020 03:37:21	Autorización	...	Firstdata authorisation s...



DOSW COMPANY

ALGUNOS CONCEPTOS A RECORDAR





INDEXACION DE BASE DE DATOS

Un índice en base de datos permite que una consulta recupere datos de una base de datos de manera eficiente, es decir permite colocar un orden en una tabla para maximizar la eficiencia buscando un registro

Cuando una tabla de base de datos no está indexada, no habrá un orden claro de las filas, por lo tanto, para completar cualquier consulta, deberá buscar en las filas de forma lineal, es decir, la consulta deberá buscar en cada fila para encontrar las filas con la condición coincidente.

INDEXACION DE BASE DE DATOS

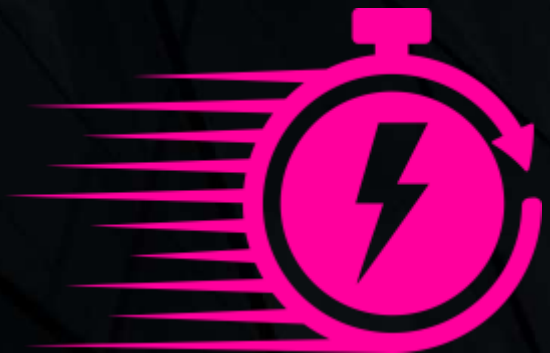
ID DE LA COMPAÑÍA UNIDAD COSTO_UNITARIO		
10	12	1.15
12	12	1.05
14	18	1.31
18	18	1.34
11	24	1.15
dieciséis	12	1.31
10	12	1.15
12	24	1.3
18	6	1.34
18	12	1.35
14	12	1,95
21	18	1.36
12	12	1.05
20	6	1.31
18	18	1.34
11	24	1.15
14	24	1.05

```
SELECT company_id, units, unit_cost FROM index_test WHERE company_id = 18
```



```
SELECT company_id, units, unit_cost FROM index_test WHERE company_id = 18
```

Con un índice en la columna
company_id



INDEXACION DE BASE DE DATOS

ID DE LA COMPAÑÍA PUNTERO	
10	_123
10	_129
11	_127
11	_138
12	_124
12	_130
12	_135
14	_125
14	_131
14	_133
dieciséis	_128
18	_126
18	_131
18	_132
18	_137
20	_136
21	_134

Los índices de la base de datos almacenan punteros para simplemente hacer referencia a la información para la ubicación de la información adicional en la memoria. En otras palabras, el índice contiene el company_id y la dirección de esa fila en particular en la memoria. En este ejemplo, el índice de la base de datos se verá así



Es como tener organizada tu despensa para poder buscar los productos que necesitas de manera mas rapida y eficaz

¿CUANDO USARLO?

Al aumentar el rendimiento de las bases de datos, se puede usar siempre que necesitemos esa mejora, entre mas grande la base de datos mas probable utilizar la indexacion



Sin embargo el indice ocupa espacio, entre mayor sea la tabla mayor va a ser el indice

Si hacemos una operacion de agregar, eliminar o actualizar, se debe hacer lo mismo en el indice



¿CUANDO NO USARLO?



Cuando se escriben datos en la base de datos, primero se actualiza la tabla original, seguida de otros índices basados en esa tabla. Cuando se realiza una escritura en la base de datos, los índices se vuelven inoperables hasta que se actualizan. Los índices nunca serán funcionales si la base de datos recibe escrituras continuamente.



COMO TRANSFORMO MI INFORMACION INICIAL EN TABLAS RELACIONALES?

Para organizar los datos en una base de datos es necesario incluir como se veria esta informacion en tablas y las relaciones entre ellas

A ESO SE LE LLAMA NORMALIZACION

1FN: Eliminar grupos repetitivos

2FN: Eliminar datos redundantes

3FN: Eliminar columnas no depende de clave

4FN: Aislar Relaciones multiples independientes

5FN: Aislar relaciones semanticamente relacionadas multiples

SOLO VAMOS A RECORDAR
3

NORMALIZACION

Si tenemos esta BD como podremos normalizarla?

Datos de facturas

número factura	fecha	cliente	número cliente	dirección	ítem factura	artículo	código artículo	unidades	precio
335	01.08.2022	Facundo Rodríguez	21	Calle Rivero 41, 41004 Sevilla	1	Teclado inalámbrico	2-0023-D	10	245 euros
335	01.08.2022	Facundo Rodríguez	21	Calle Rivero 41, 41004 Sevilla	2	Cable red UTP RJ45	4-0023-D	12	90 céntimos
335	01.08.2022	Facundo Rodríguez	21	Calle Rivero 41, 41004 Sevilla	3	Cámara web USB	5-0023-D	1	165 euros
336	02.08.2022	Natalia Martínez	22	Calle San Eloy 82, 41001 Sevilla	1	Auriculares inalámbricos	1-0023-D	2	175 euros
336	02.08.2022	Natalia Martínez	22	Calle San Eloy 82, 41001 Sevilla	2	Mini Adaptador USB-C USB 2.0	3-0023-D	2	75 céntimos
337	02.08.2022	Carlos García	23	Avenida de Italia 245, 21003, Huelva	1	Teclado inalámbrico	2-0023-D	25	245 euros



NORMALIZACION 1FN

Para iniciar el proceso de normalización de esta tabla y lograr la primera forma normal debemos:

- Dividir todos los datos con “multi-valor” en columnas separadas (datos atómicos) y comprobar que los valores en cada columna sean del mismo tipo.

número factura	fecha	apellido	nombre	número cliente	calle	número	código postal	municipio	ítem factura	artículo	código artículo	unidades	precio
335	01.08.2022	Rodríguez	Facundo	21	Calle Rivero	41	41004	Sevilla	1	Teclado inalámbrico	2-0023-D	10	245
335	01.08.2022	Rodríguez	Facundo	21	Calle Rivero	41	41004	Sevilla	2	Cable red UTP RJ45	4-0023-D	12	0.90
335	01.08.2022	Rodríguez	Facundo	21	Calle Rivero	41	41004	Sevilla	3	Camara web USB	5-0023-D	1	165
336	02.08.2022	Martínez	Natalia	22	Calle San Eloy	82	41001	Sevilla	1	Auriculares inalámbricos	1-0023-D	2	175
336	02.08.2022	Martínez	Natalia	22	Calle San Eloy	82	41001	Sevilla	2	Mini Adaptador USB-C USB 2.0	3-0023-D	2	0.75
337	02.08.2022	García	Carlos	23	Avenida de Italia	245	21003	Huelva	1	Teclado inalámbrico	2-0023-D	25	245

NORMALIZACION 2FN

Para estar en la segunda forma normal, a las condiciones de la primera se añade las siguientes:

- Los atributos que no forman parte de ninguna clave han de depender funcionalmente de toda la clave primaria.
- Comprobamos que todos los atributos no-clave dependen por completo de la clave primaria. Esta dependencia existe si todos los atributos de la clave primaria son necesarios para identificar a los atributos no-clave.
- Relegamos a tablas diferentes los atributos no-clave que no dependen de la clave primaria.

Facturas



número factura	fecha	apellido	nombre	número cliente	calle	número	código postal	municipio
335	01.08.2022	Rodríguez	Facundo	21	Calle Rivero	41	41004	Sevilla
336	02.08.2022	Martínez	Natalia	22	Calle San Eloy	82	41001	Sevilla
337	02.08.2022	García	Carlos	23	Avenida de Italia	245	21003	Huelva



Items

número factura	Item factura	artículo	código artículo	unidades	precio
335	1	Teclado inalámbrico	2-0023-D	10	245
335	2	Cable red UTP RJ45	4-0023-D	12	0.90
335	3	Camara web USB	5-0023-D	1	165
336	1	Auriculares inalámbricos	1-0023-D	2	175
336	2	Mini Adaptador USB-C USB 2.0	3-0023-D	2	0.75
337	1	Teclado inalámbrico	2-0023-D	25	245

La 2FN elimina las llamadas dependencias parciales.

NORMALIZACION 3FN

En la tercera forma normal debe cumplir las condiciones de las dos primeras y además:

- Los atributos no-clave no pueden depender de forma transitiva de una clave candidata (cuando un atributo que no sea clave depende de una clave primaria a través de otro atributo que no sea clave).

número factura	fecha	número cliente
335	01.08.2022	21
336	02.08.2022	22
337	02.08.2022	23

FACTURA



número cliente	apellido	nombre	calle	número	código postal	municipio
21	Rodriguez	Facundo	Calle Rivero	41	41004	Sevilla
22	Martinez	Natalia	Calle San Eloy	82	41001	Sevilla
23	García	Carlos	Avenida de Italia	245	21003	Huelva

CLIENTES



número factura	ítem factura	código artículo	unidades
335	1	2-0023-D	10
335	2	4-0023-D	12
335	3	5-0023-D	1
336	1	1-0023-D	2
336	2	3-0023-D	2
337	1	2-0023-D	25

ITEMS



código artículo	artículo	precio
2-0023-D	Teclado inalámbrico	245
4-0023-D	Cable red UTP RJ45	0.90
5-0023-D	Cámara web USB	165
1-0023-D	Auriculares inalámbricos	175
3-0023-D	Mini Adaptador USB-C USB 2.0	0.75
2-0023-D	Teclado inalámbrico	245

ARTICULOS

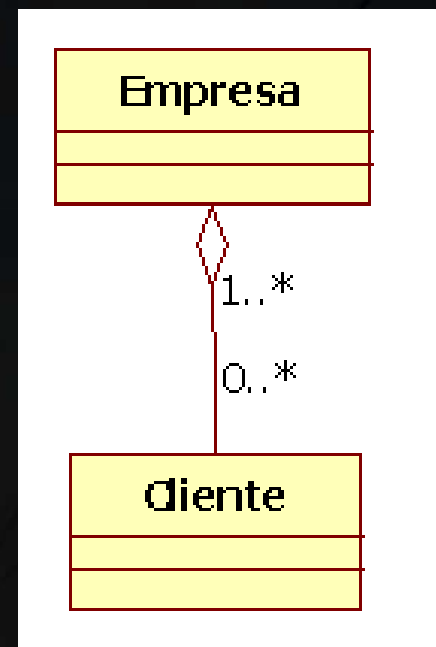


AGREGACION VS COMPOSICION

Agregacion es un tipo de asociacion que indica que una clase es parte de otra

Los componentes pueden ser compartidos por varios compuestos

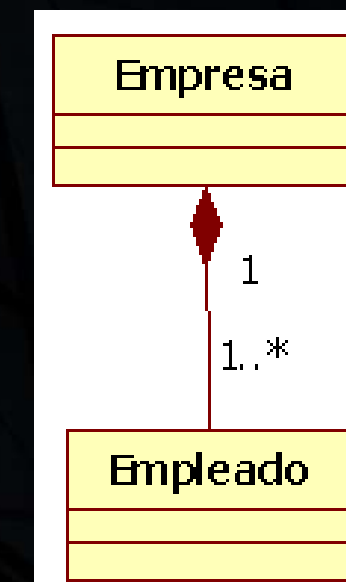
La destruccion no conlleva a la destruccion de los demas componentes



Composicion es una forma fuerte de asociacion donde la vida de la clase contenida debe coincidir con la vida de la clase contenedor

Los componentes no pueden ser compartidos por varios compuestos

La destruccion conlleva a la destruccion de los demas componentes

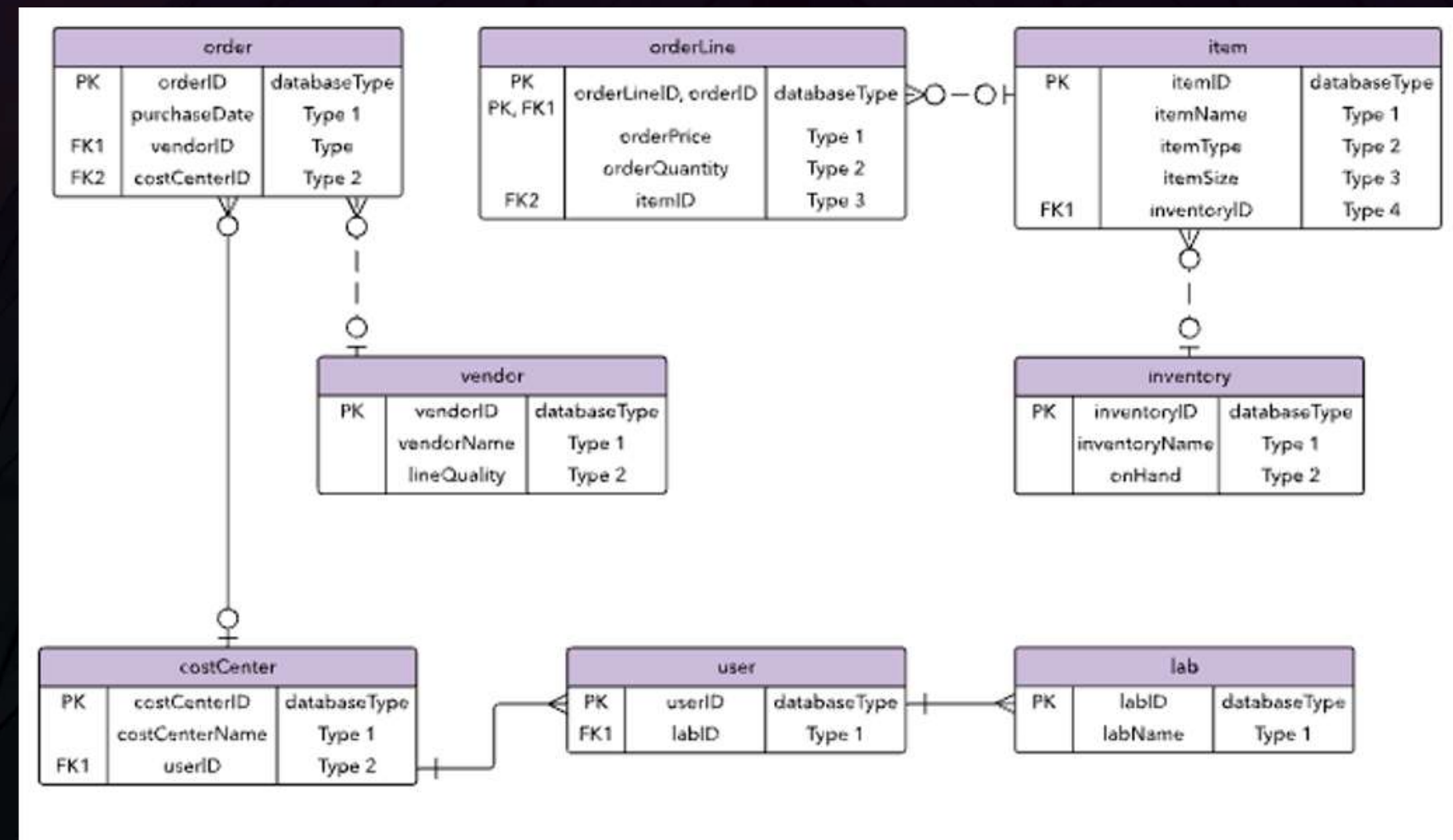




MODULO DE DIAGRAMACION BD RELACIONAL

MODELO ENTIDAD RELACION

Muestra la estructura lógica de la base, incluidas las relaciones y limitaciones que determinan como se almacenan los datos y como se accede a ellos.



El modelo relacional es el mas común, ordenando los datos en tablas, también conocidas como relaciones, cada columna enumera un atributo de la entidad, cada fila o tupla incluye datos sobre una instancia específica de la entidad



COMO SERIA EL MODELO RELACIONAL DE MI RESTAURANTE?

Revisen dentro de su idea de restaurante, que informacion es la que van a guardar en BD, como la guardarían, que tablas y relaciones necesitarían

COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI RESTAURANTE ?



Los datos van a PostgreSQL. Spring Data JPA genera las queries. Tu dominio no toca la base de datos directamente — lo hace el Repository a través de la entidad.



COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

1

Con spring initializr pueden agregar la dependencia de data en sus proyectos al momento de generarlo, con esto tendran alineada la dependencia directamente desde el inicio

A screenshot of the Spring Initializr web application. The form is dark-themed and contains several sections for configuring a new project. The "Project" section has radio buttons for "Gradle - Groovy", "Gradle - Kotlin", "Maven" (selected), and "Groovy". The "Language" section has radio buttons for "Java" (selected), "Kotlin", and "Groovy". The "Spring Boot" section has radio buttons for "4.0.0 (SNAPSHOT)", "4.0.0 (M2)", "3.5.6 (SNAPSHOT)", "3.5.5" (selected), and "3.4.10 (SNAPSHOT)". The "Project Metadata" section has input fields for "Group" (com.example), "Artifact" (demo), "Name" (demo), "Description" (Demo project for Spring Boot), and "Package name" (com.example.demo). The "Packaging" section has radio buttons for "Jar" (selected) and "War". The "Dependencies" section has a button "ADD DEPENDENCIES... % + B" and two listed dependencies: "Spring Data JPA" (SQL) and "H2 Database" (SQL).

spring initializr

Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Maven ☐ Groovy

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.0.0 (SNAPSHOT) ☐ 4.0.0 (M2) ☐ 3.5.6 (SNAPSHOT) ☒ 3.5.5 ☐ 3.4.10 (SNAPSHOT) ☐ 3.4.9

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging

☒ Jar ☐ War

Java ☐ 24 ☐ 21 ☒ 17

Dependencies [ADD DEPENDENCIES... % + B](#)

Spring Data JPA [SQL](#)
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

H2 Database [SQL](#)
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.



COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

1

Si no, pueden agregar la dependencia en su Maven directamente

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

RECUERDEN: Tienen que tener el spring-boot-starter-web tambien agregado como dependencia

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Si necesitan una Base de datos embebida pueden agregar H2, si quieren la conexion directa con Postgresql tambien lo agregan como dependencias

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

2

Una vez tengamos las dependencias en nuestro proyecto, es momento de definir las entidades de nuestra aplicacion

```
// persistence/entity/PlatoEntity.java
// Esta clase representa la tabla "platos" en PostgreSQL
// No tiene lógica de negocio - solo estructura y anotaciones JPA
@Entity
@Table(name = "platos")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class PlatoEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // la BD asigna el ID
    private Long id;

    @Column(nullable = false, unique = true, length = 120)
    private String nombre;

    @Column(nullable = false, precision = 10, scale = 2)
    private Double precio;

    @Column(nullable = false, length = 60)
    private String categoria;

    private String descripcion;

    @Column(nullable = false)
    private Boolean disponible;

    @CreationTimestamp // Hibernate la llena automáticamente
    private LocalDateTime creadoEn;
}
```



Una Entidad es la manera en la que nombramos una entidad de software que representa una tabla a nivel de código de una estructura relacional



Usamos la anotación de ejemplo PlatoEntity en código para identificar el tipo de objeto



Una Entidad también puede utilizar las anotaciones de Lombok así nos liberamos de realizar manualmente ciertos métodos de clase

DE DONDE SALEN LAS ENTIDADES?

Del diagrama Entidad relacion que realizamos previamente, La entidad representa la tabla en la base de datos. Su trabajo es persistencia — no tiene lógica de negocio. Vive en `persistence/entity/` para dejar claro que es una clase de infraestructura.

✗ Anti-patrón — mezclar dominio y entidad

- ✗ Una sola clase con `@Entity` y lógica de negocio
- ✗ La BD dicta la estructura del dominio
- ✗ Imposible cambiar el esquema sin romper lógica
- ✗ Las anotaciones JPA contaminan el modelo de negocio

✓ Correcto — separar dominio de entidad

- ✓ `model/domain/Plato.java` — tiene lógica
- ✓ `persistence/entity/PlatoEntity.java` — tiene anotaciones JPA
- ✓ El Mapper traduce entre ambas
- ✓ Cambiar el esquema no toca el dominio

CREA LAS ENTIDADES Y SUS RELACIONES EN CODIGO



Recuerda Utilizar las anotaciones correctas - @Entity,
@Table, @GeneratedValue, @Column, etc

COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

3

Creamos nuestros Repositories y lo extendemos de la Interface que nos ofrece Spring Data

```
// repository/PlatoRepository.java
@Repository
public interface PlatoRepository extends JpaRepository<PlatoEntity, Long> {

    // Spring Data genera el SQL automáticamente por el nombre del método
    List<PlatoEntity> findByDisponibleTrue();

    List<PlatoEntity> findByCategoriaIgnoreCase(String categoria);

    boolean existsByNombreIgnoreCase(String nombre);

    boolean existsByNombreIgnoreCaseAndIdNot(String nombre, Long id);
}
```



CRUD REPOSITORY y JpaRepository son las interfaces que nos ofrece Spring data para realizar las operaciones que necesitamos



Es necesario colocar primero la entidad que representa la tabla y luego el tipo del identificador que tiene ese objeto (debe ser el mismo que se establece en el objeto dao)

```
@Id
@GeneratedValue(strategy=GenerationType.AUTO)
private Long id;
```

JpaRepository ya te da: save(), findById(), findAll(), deleteById(), existsById(), count() — sin escribir una línea de SQL. Los métodos que agregas siguen la convención de nombres de Spring Data y también se generan automáticamente.

CREA LOS REPOSITARIOS NECESARIOS



Recuerda es SOLO 1 repositorio por cada entidad que creaste



COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

```
List<User> findByNameAndActive(String name, boolean active);
```

Conceptualmente termina haciendo algo similar a:

```
SELECT *  
FROM users  
WHERE name = ?  
AND active = ?
```



Podemos tambien utilizar queries generadas de manera manual por nosotros, normalmente con @Query, usando JPQL o SQL nativo – El beneficio principal es tener mayor control cuando la consulta deja de ser trivial.

```
@Query("SELECT f FROM Foo f WHERE LOWER(f.name) = LOWER(:name)")  
Foo retrieveByName(@Param("name") String name);
```

COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

4

En el application.properties o en el application.yaml configuremos las variables

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mi_bd?useSSL=false&serverTimezone=UTC
    username: root
    password: 1234
    driver-class-name: com.mysql.cj.jdbc.Driver

  jpa:
    hibernate:
      ddl-auto: update # opciones: none, validate, update, create, create-drop
    show-sql: true
    properties:
      hibernate:
        format_sql: true
    database-platform: org.hibernate.dialect.MySQL8Dialect

server:
  port: 8080
```

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/mi_bd
    username: postgres
    password: 1234
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
    database-platform: org.hibernate.dialect.PostgreSQLDialect

server:
  port: 8080
```

ddl-auto: update es para desarrollo. Nunca uses create en producción — borra y recrea las tablas. En producción se usan migraciones (Liquibase / Flyway). Por ahora, update está bien.

COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

- 5 Creamos un servicio para manejar la logica que es el encargado de usar el repositorio y un controlador REST que es el que llama nuestro servicio

```
package com.ejemplo.demo.controller;

import com.ejemplo.demo.model.Usuario;
import com.ejemplo.demo.service.UsuarioService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/usuarios")
public class UsuarioController {

    private final UsuarioService service;

    public UsuarioController(UsuarioService service) {
        this.service = service;
    }

    @GetMapping
    public List<Usuario> listarUsuarios() {
        return service.obtenerUsuarios();
    }

    @PostMapping
    public Usuario crearUsuario(@RequestBody Usuario usuario) {
        return service.guardarUsuario(usuario);
    }

    @GetMapping("/{email}")
    public Usuario buscarPorEmail(@PathVariable String email) {
        return service.buscarPorEmail(email);
    }
}
```

```
package com.ejemplo.demo.service;

import com.ejemplo.demo.model.Usuario;
import com.ejemplo.demo.repository.UsuarioRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class UsuarioService {

    private final UsuarioRepository repository;

    public UsuarioService(UsuarioRepository repository) {
        this.repository = repository;
    }

    public List<Usuario> obtenerUsuarios() {
        return repository.findAll();
    }

    public Usuario guardarUsuario(Usuario usuario) {
        return repository.save(usuario);
    }

    public Usuario buscarPorEmail(String email) {
        return repository.findByEmail(email);
    }
}
```

ES MOMENTO DE USAR LA PERSISTENCIA REAL

5 En el caso de ustedes, reemplacen lo que usaron para guardar datos en memoria por el llamado a sus repositorios



Van a inyectar el repositorio en el servicio

Van a crear un mapper que permita transformar los objetos del dominio en entidades de persistencia

Modifiquen los metodos para que soporten lo que el repositorio les retorna –
SIGAN USANDO STREAMS



ES MOMENTO DE USAR LA PERSISTENCIA REAL

- ★ Los mappers traducían entre RequestDTO y dominio, y entre dominio y ResponseDTO. En esta semana agregamos una tercera traducción: dominio ↔ entidad JPA. Cada dirección tiene un nombre claro.



```
// Dominio → Entidad JPA (para guardar en BD)
// El id se ignora en creación - la BD lo asigna
@Mapping(target = "creadoEn", ignore = true) // @CreationTimestamp lo llena Hibernate
PlatoEntity toEntity(Plato plato);

// Entidad JPA → Dominio (al leer de BD)
Plato toDomain(PlatoEntity entidad);
```



COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?



Normalmente no necesitamos usar `@EnableJpaRepositories` ya que Springboot lo hace ,automaticamente si los repositorios estan en paquetes hijos del paquete principal, si no estan `@EnableJpaRepositories(basePackages = "com.ejemplo.demo.repository")` <-- opcional si tus repositorios están fuera del paquete base

```
package com.ejemplo.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
// @EnableJpaRepositories(basePackages = "com.ejemplo.demo.repository")
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

- 6** Es momento de Probar la API, Recuerda que antes de todo tu Base de datos ya debe estar construida, si usas MySQL o Postgres es necesario que ya este creada y con las configuraciones que estableciste – Ejecuta cada operacion para 1 entidad y verifica como se ve en BD



CREALA



LISTALA



ACTUALIZALE
UN DATO



BORRALA

ALGUNAS ANOTACIONES ADICIONALES

@ManyToOne : Muchas filas de una tabla apuntan a una sola fila de otra

EJEMPLO

- Muchas órdenes pertenecen a un usuario

```
@Entity
public class Order {

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;
}
```

- Siempre lleva `@JoinColumn`
- Es el lado que controla la relación

@OneToMany : Una fila se relaciona con muchas en otra tabla

EJEMPLO

- Un usuario tiene muchas órdenes

```
@Entity
public class User {

    @Id
    @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "user") // hace referencia al campo en Order
    private List<Order> orders;
}
```

- `mappedBy` es obligatorio si ya existe el `@ManyToOne`
- No crea columna nueva

ALGUNAS ANOTACIONES ADICIONALES

@OneToOne : Una Fila se relaciona con exactamente una columna

EJEMPLO

- Un usuario tiene un perfil

```
@Entity
public class User {

    @Id
    @GeneratedValue
    private Long id;

    @OneToOne
    @JoinColumn(name = "profile_id")
    private Profile profile;
}
```

- La FK puede estar en cualquiera de las dos tablas
- Usualmente lleva `UNIQUE` en DB
- Cuidado con `cascade`

@ManyToMany : Muchas filas se relacionan con muchas

EJEMPLO

- Un estudiante tiene muchos cursos
- Un curso tiene muchos estudiantes

```
@Entity
public class Student {

    @Id
    @GeneratedValue
    private Long id;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private List<Course> courses;
}
```

```
@Entity
public class Course {

    @Id
    @GeneratedValue
    private Long id;

    @ManyToMany(mappedBy = "courses")
    private List<Student> students;
}
```

- En sistemas reales, muchas veces se reemplaza por entidad intermedia (ej: `Enrollment`)

- Siempre hay tabla intermedia
- `@JoinTable` define esa tabla



ALGUNAS ANOTACIONES ADICIONALES

1. Si las van a utilizar utilicen siempre como fetch type LAZY
2. Eviten usar Many To Many en sistemas complejos, es mejor usar una entidad intermedia
3. Nunca olviden sincronizar las dos tablas - Bilateral

QUE ES FETCH TYPE

Es la manera en la que menciono a la bd si cargo los datos relacionados de una vez o solo cuando los necesite, existen diferentes tipos:

LAZY

Carga los datos solo cuando los usas

EAGER

Carga los datos inmediatamente

ALGUNAS ANOTACIONES ADICIONALES

EAGER puede afectar rendimiento sin que lo noten, LAZY puede lanzar errores si no se maneja bien la sesion.

PROBLEMAS COMUNES

1+N Query Problem:

```
List<Order> orders = repo.findAll();  
  
for (Order o : orders) {  
    o.getUser(); // dispara una query por cada order  
}
```

1 Query a orders
N Queries Users

Solucion con JOIN FETCH

```
@Query("SELECT o FROM Order o JOIN FETCH o.user")  
List<Order> findAllWithUser();
```

```
SELECT o.*, u.*  
FROM orders o  
JOIN users u ON o.user_id = u.id;
```

Solucion con EntityGraph

```
@EntityGraph(attributePaths = {"user"})  
List<Order> findAll();
```



PREGUNTAS DE ATENCION

¿Por qué PlatoEntity y Plato son clases distintas?

Porque tienen responsabilidades distintas. Plato tiene comportamiento de negocio (estaDisponible, activar). PlatoEntity tiene anotaciones de persistencia (@Column, @GeneratedValue). Si las mezclas, cambiar el esquema de la BD impacta la lógica de negocio — dos razones de cambio en una clase.

¿Quién asigna el ID ahora — el código o la BD?

La base de datos, con @GeneratedValue(strategy = IDENTITY). Por eso desaparece el AtomicLong del Service. El ID está disponible después del save() — el repositorio lo retorna en el objeto guardado.

¿Por qué el Validator ya no recibe la colección completa de platos?

Porque ya no tiene sentido cargar todos los platos en memoria para buscar duplicados. El Validator inyecta el Repository y llama existsByNombreIgnoreCase() — una query que el motor de base de datos ejecuta de forma óptima, con índice en la columna.

BASES DE DATOS NO RELACIONALES

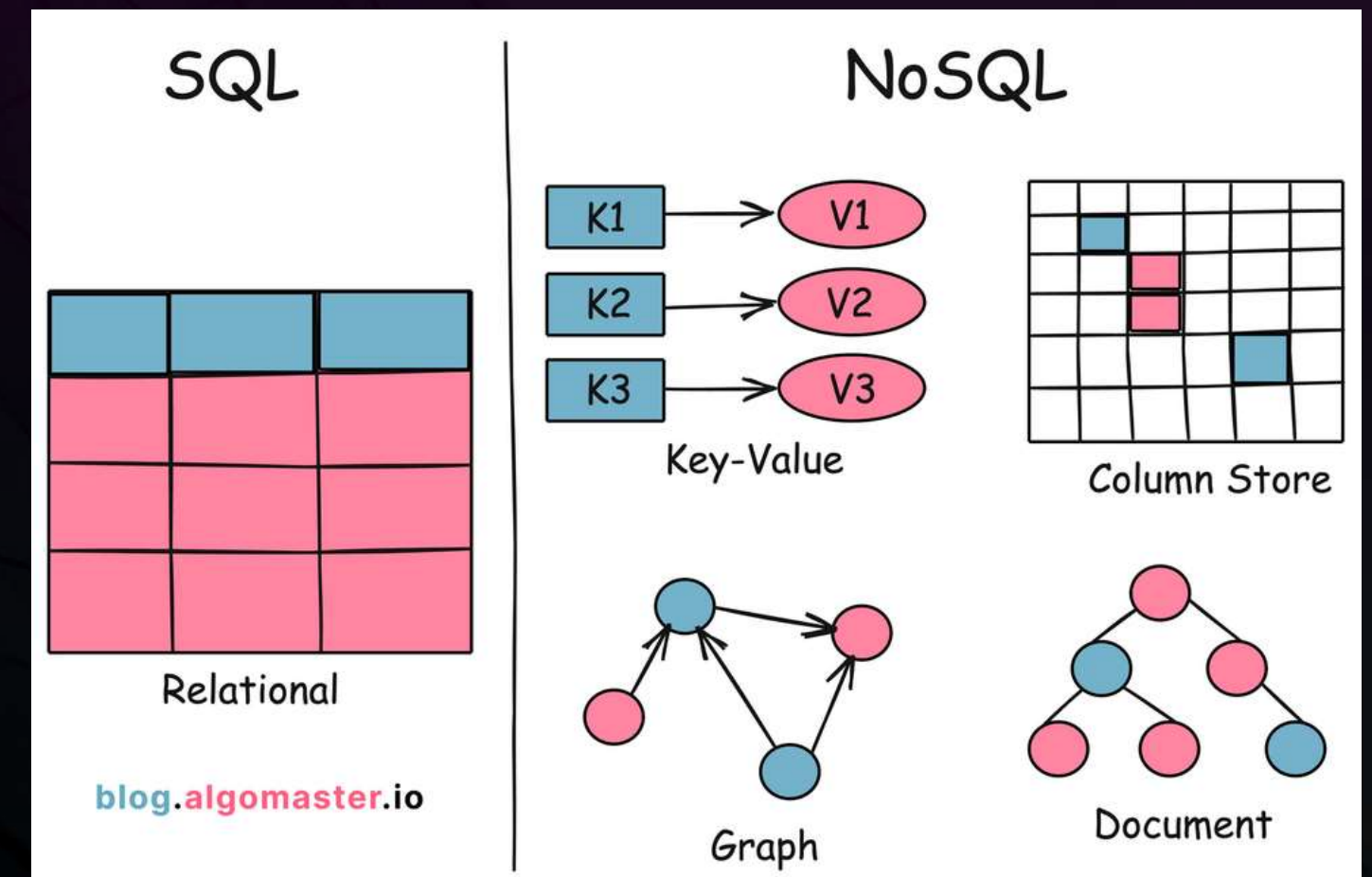


NOSQL VS SQL

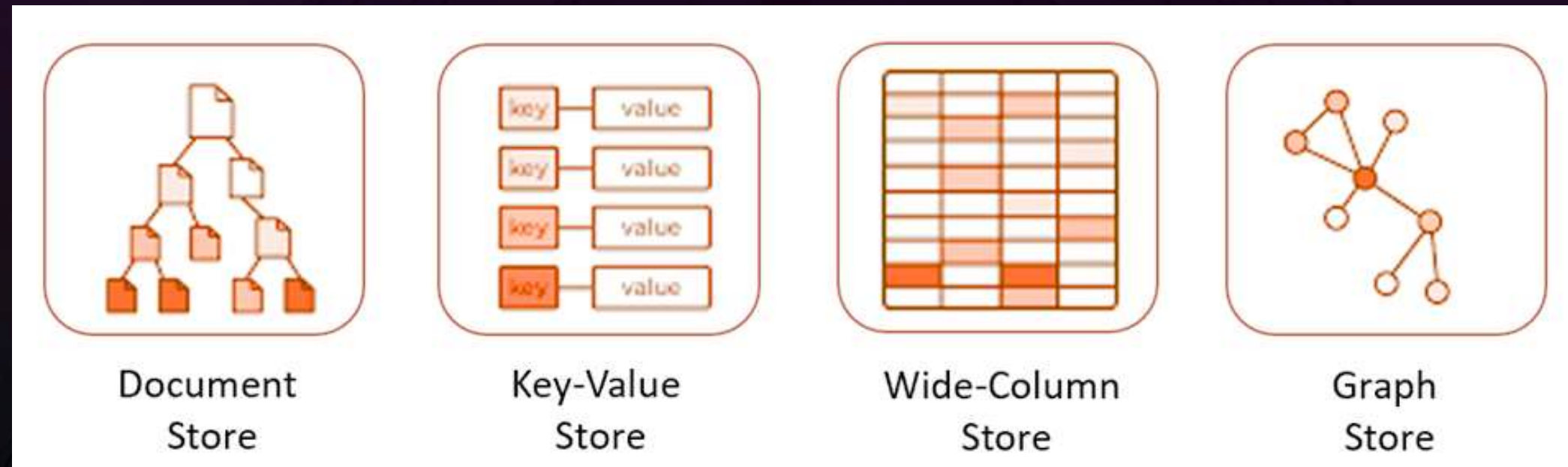
Son bases de datos no relacionales que almacenan datos de manera diferente a las relaciones tabulares utilizadas en las bases de datos SQL.

Mientras que las bases de datos SQL son ideales para datos estructurados, las bases de datos NoSQL son adecuadas para datos estructurados, semiestructurados y no estructurados.

Como resultado, las bases de datos NoSQL no siguen un esquema rígido, sino que tienen estructuras más flexibles para adaptarse a sus tipos de datos.



VENTAJAS Y DESVENTAJAS DE NOSQL



Esquema flexible
Utilizable en plataformas de infraestructura distribuida
Infraestructura de bajo costo
Alta disponibilidad y rendimiento

Tecnología menos madura y difícil de gestionar
Capacidades de consulta limitadas
Inconsistencia de datos y bajo rendimiento en algunos escenarios complejos

PERO COMO GARANTIZAMOS LA TRANSACCIONALIDAD ?

En base de datos relacional no existe

A.C.I.D





BA.S.E

Es el acronimo de Básicamente Disponible, Estado Suave, Consistencia Eventual. Define un conjunto de propiedades diseñadas para bases de datos NOSQL, priorizando la disponibilidad y la flexibilidad sobre la consistencia estricta

BASICAMENTE DISPONIBLE

el sistema garantiza la disponibilidad, lo que significa que responderá a las solicitudes incluso si algunas partes del sistema están inactivas o no se pueden acceder.

ESTADO SUAVE

debido a actualizaciones asincrónicas, el estado del sistema puede cambiar con el tiempo, incluso sin entrada.

CONSISTENCIA EVENTUAL

los datos eventualmente se volverán consistentes, pero puede haber períodos en los que sean inconsistentes temporalmente.



MONGO DB

Es una base de datos de tipo documento, que almacena los datos en un formato JSON llamado BSON

Un registro en Mongo es un documento, que es una estructura de datos compuesta por pares clave.valor similar a una estructura de los objetos JSON

```
{  
  title: "Post Title 1",  
  body: "Body of post.",  
  category: "News",  
  likes: 1,  
  tags: ["news", "events"],  
  date: Date()  
}
```



EMBEBIDO VS REFERENCIADO

Embedded vs. Referenced

Key: Brewery_Pike_Brewing
{ "Name": "Pike Brewing", ... }
"Brewery_Address": { ... City: Seattle Country: USA }
}

Key: Brewery_Pike_Brewing
{ "Name": "Pike Brewing", "Brewery_Address_ID": "Brewery_Address_Pike_Brewing", ... }

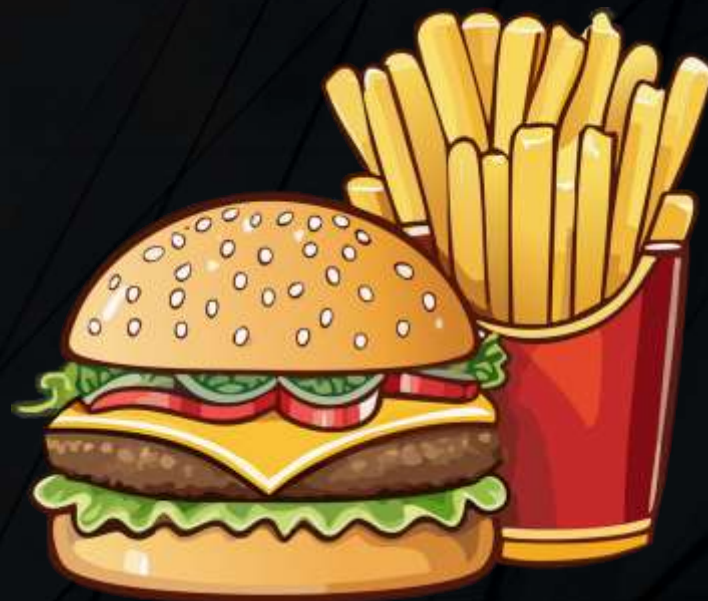
Key: Brewery_Address_Pike_Brewing
"Brewery_Address": { ... City: Seattle Country: USA }

EMBEBIDO VS REFERENCIADO

En el modelo embebido se incrusta la informacion relacionada, como lo son los detalles de un pedido y sus articulos, se incluye todo dentro de un solo documento

Permite obtener toda la informacion ya que se realiza con una unica operacion de lectura

Puede tener duplicacion de informacion



Los documentos estan separados y se establece una relacion mediante la insercion de un campo con el _id de un documento en otro

Permite mayor flexibilidad para modelar diferentes tipos de relaciones pero requiere de multiples consultas

Evita la duplicacion y redundancia de los datos al tener la informacion en un solo lugar



CUANDO USAR CADA UNO?

Si en el diseño del Diagrama de Clases nuestros Objetos tienen una relacion de 1 a pocos y los datos no cambian o siempre se consultan juntos

EMBEBIDO

```
pedidos
{
  _id: ObjectId("64f..."),
  mesa: "5",
  estado: "En preparación",
  items: [
    { nombre: "Hamburguesa", cantidad: 1, precio: 25000 },
    { nombre: "Papas", cantidad: 1, precio: 8000 }
  ],
  total: 33000,
  fecha: ISODate("2026-09-21T20:15:00Z")
}
```

Si en el diseño del Diagrama de Clases nuestros Objetos tienen una relacion de muchos a muchos, hay mucha reutilizacion o la informacion es de gran tamaño

REFERENCIADO

```
pedidos
{
  _id: ObjectId("64f..."),
  mesa: "5",
  estado: "En preparación",
  items: [
    { menuItemId: ObjectId("a1..."), cantidad: 1 },
    { menuItemId: ObjectId("b2..."), cantidad: 1 }
  ],
  total: 33000,
  fecha: ISODate("2026-09-21T20:15:00Z")
}

menuItems
{
  _id: ObjectId("a1..."),
  nombre: "Hamburguesa",
  precio: 25000,
  categoria: "Platos fuertes"
}
{
  _id: ObjectId("b2..."),
  nombre: "Papas",
  precio: 8000,
  categoria: "Acompañamientos"
}
```

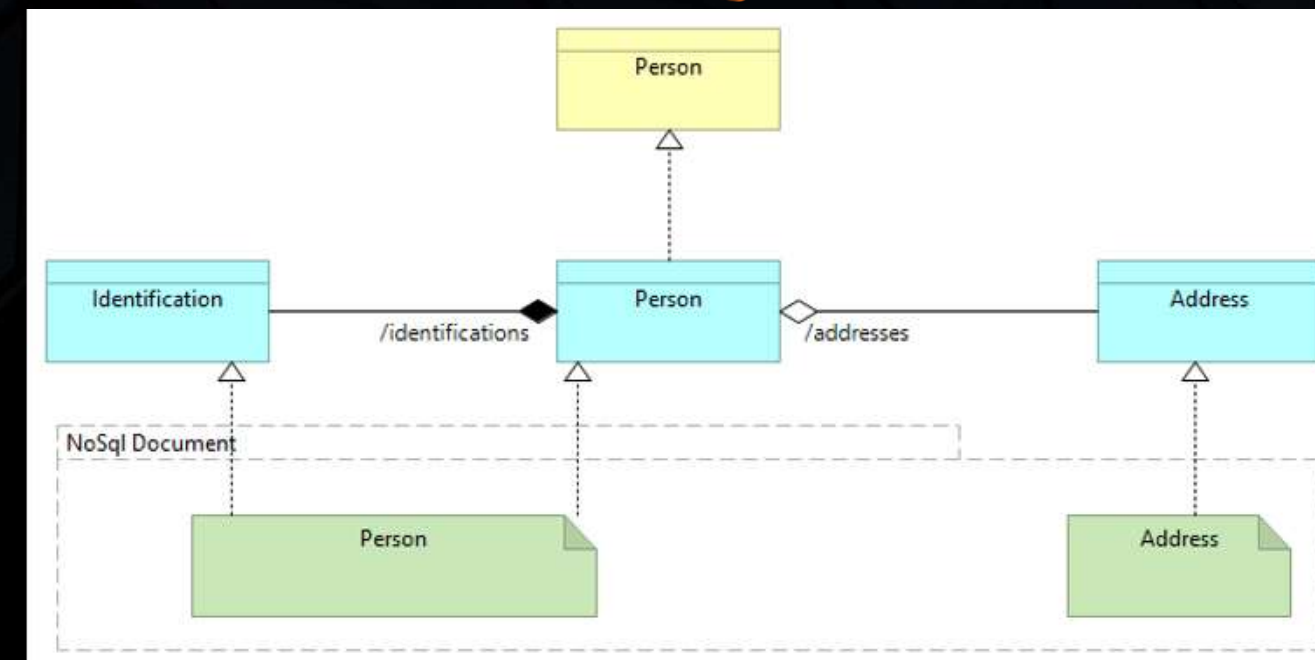
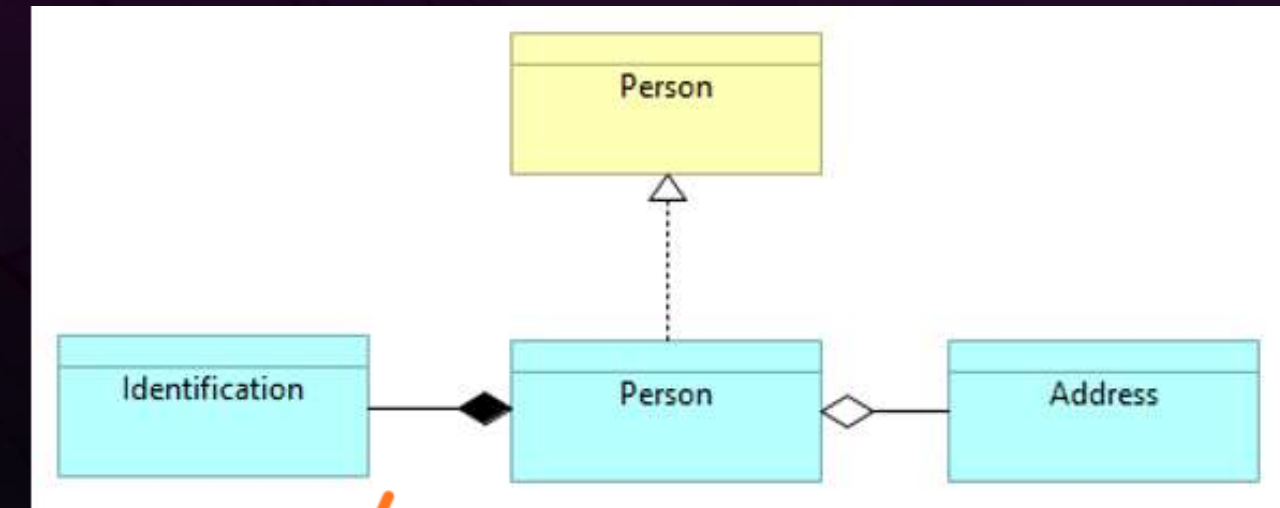


DOSW COMPANY

MODULO DE DIAGRAMACION BD RELACIONAL

MODELO DE DOCUMENTOS

- El esquema puede variar dinámicamente agregando campos y conceptos en tiempo de ejecución y variar entre las distintas particiones de almacenamiento.
- Es necesario que identifiquen lo que quieren diseñar y como se estructura su objeto.
- Hay 3 propuestas para poderlas modelar:
 - **Modelo y meta-modelo conceptual**
 - Meta-modelo lógico
 - Modelo y meta-modelo físico



COMO SERIA EL MODELO NO RELACIONAL DE MI RESTAURANTE?



Revisen dentro de su idea de restaurante como transformarian su solucion pero en un modelo NOSQL como guardarian la informacion? cuales documentos generarian? Embebido o referenciado?

COMO IMPLEMENTAR UNA BASE DE DATOS NO RELACIONAL EN MI RESTAURANTE ?

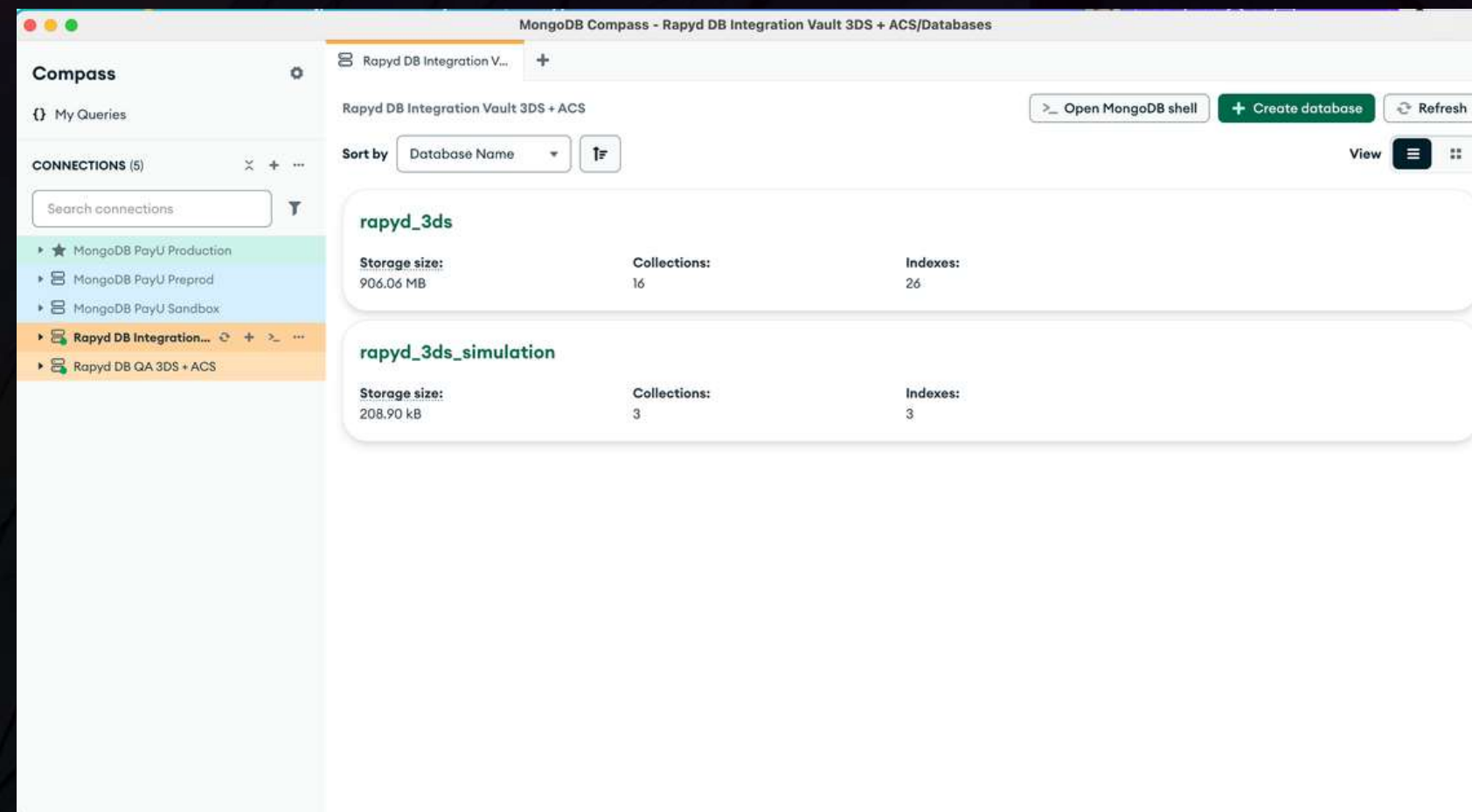


Algunos dominios del restaurante encajan mejor en documentos: registros de auditoría, logs de eventos, catálogos con estructura variable. El patrón es el mismo — Documento en vez de Entity, MongoRepository en vez de JpaRepository.

COMO IMPLEMENTAR UNA BASE DE DATOS NO RELACIONAL EN MI API?

1

Configurar Mongo DB y utilizar MongoDB Compass



Sign up

See what Atlas is capable of for free

[Sign up with Google](#)

First Name* Last Name*

Company

Email*

Password*

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#).

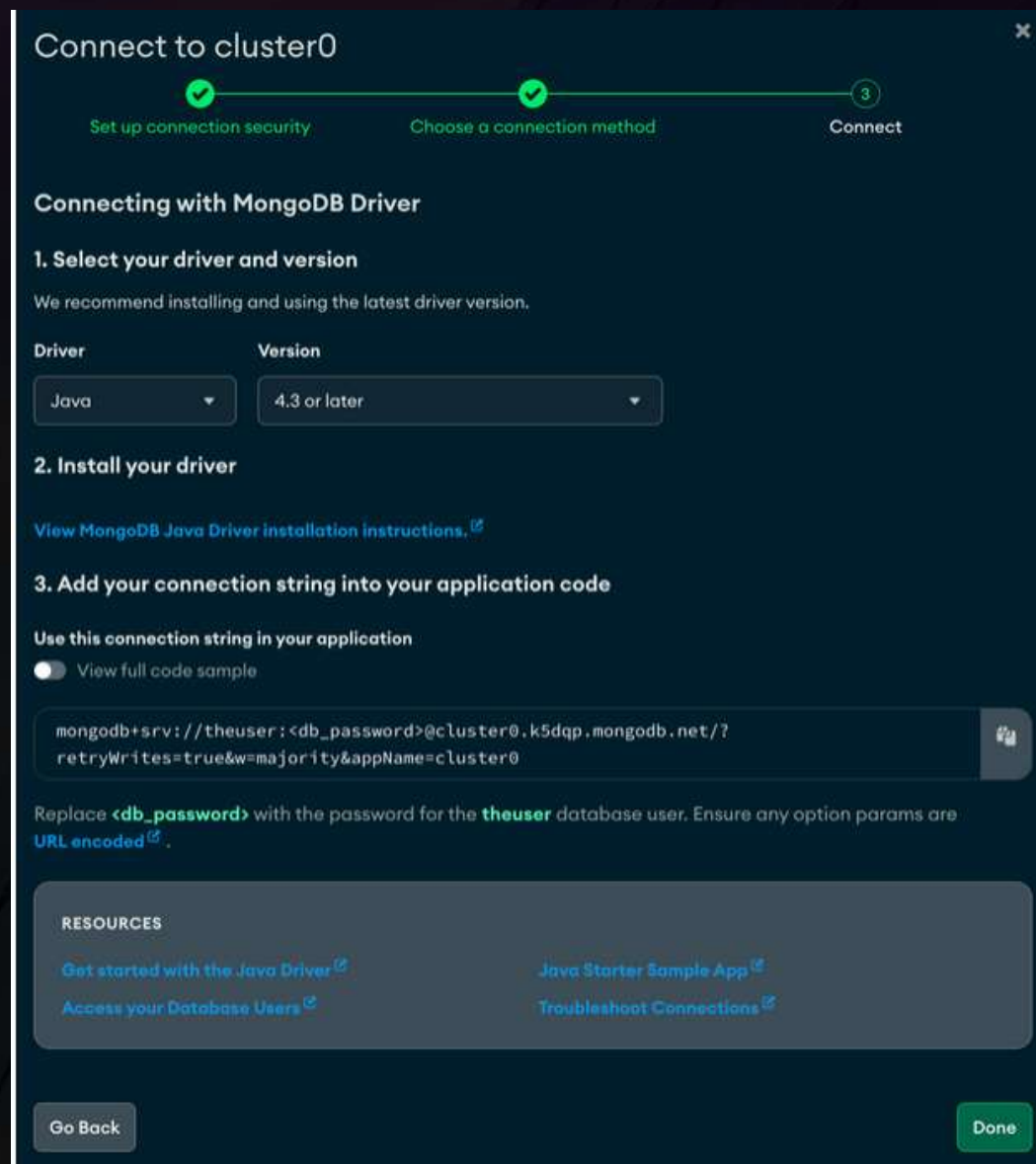
[Create your Atlas account](#)

[Sign in](#)

COMO IMPLEMENTAR UNA BASE DE DATOS NO RELACIONAL EN MI API?

2

Establecer la conexión con el cluster de mongo y generar el proyecto con su dependencia



Connect to cluster0

Set up connection security Choose a connection method Connect

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver Version

Java 4.3 or later

2. Install your driver

[View MongoDB Java Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☐ View full code sample

mongodb+srv://theuser:<db_password>@cluster0.k5dqp.mongodb.net/?retryWrites=true&w=majority&appName=cluster0

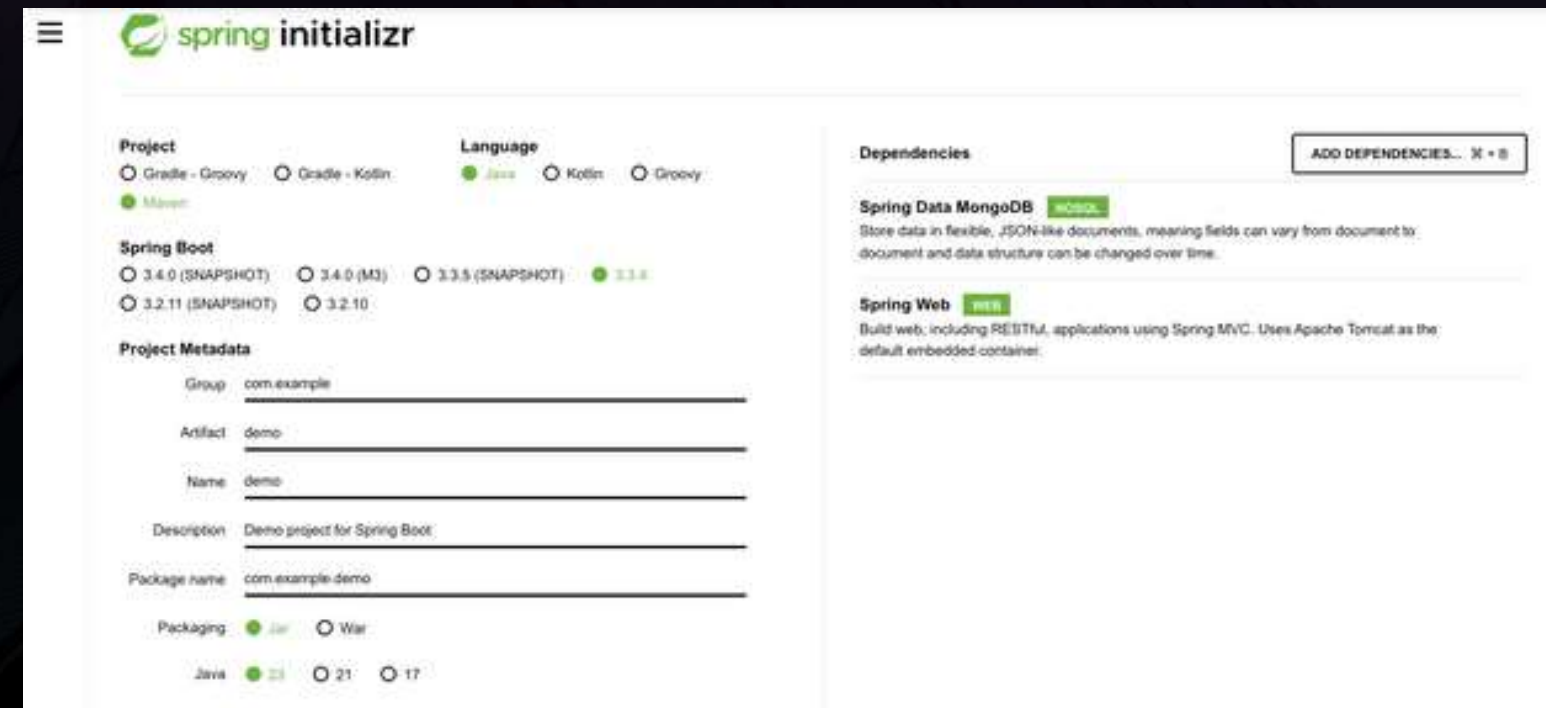
Replace <db_password> with the password for the theuser database user. Ensure any option params are URL encoded.

RESOURCES

[Get started with the Java Driver](#) [Java Starter Sample App](#)

[Access your Database Users](#) [Troubleshoot Connections](#)

Go Back Done



spring initializr

Project Language

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Java ☐ Kotlin ☐ Groovy

☒ Maven

Spring Boot

☐ 3.4.0 (SNAPSHOT) ☐ 3.4.0 (M3) ☐ 3.3.5 (SNAPSHOT) ☒ 3.3.4

☐ 3.2.11 (SNAPSHOT) ☐ 3.2.10

Project Metadata

Group com.example

Artifact demo

Name demo

Description Demo project for Spring Boot

Package name com.example.demo

Packaging ☒ Jar ☐ War

Java ☒ 23 ☐ 21 ☐ 17

Dependencies

[ADD DEPENDENCIES...](#)

Spring Data MongoDB [docs](#)

Store data in flexible, JSON-like documents, meaning fields can vary from document to document and data structure can be changed over time.

Spring Web [docs](#)

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.



COMO IMPLEMENTAR UNA BASE DE DATOS NO RELACIONAL EN MI API?

3

En el application.yaml configuremos las variables que obtuvimos en el paso anterior

```
spring:
  data:
    mongodb:
      uri: mongodb+srv://<username>:<pwd>@<cluster>.mongodb.net/mygrocerylist
      database: mygrocerylist
```

Recuerda el username es el que tu ingresaste, el pwd es la contraseña para esa base de datos correspondiente, en el caso de ustedes mygrocerylist debe cambiar al nombre de su restaurante

COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

4

Es momento de definir las entidades de persistencia en este caso los documentos, para eso nos basamos en el diagrama de documentos que creamos

```
@Document(collection = "eventos_restaurante")
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class EventoRestauranteDocument {

    @Id
    private String id; // Mongo usa String por defecto (ObjectId en BSON)

    private String tipo; // "PEDIDO_CREADO", "PLATO_AGOTADO", etc.
    private String entidadTipo; // "Pedido", "Plato", "Mesa"
    private Long entidadId;
    private String descripcion;
    private String usuario;
    private LocalDateTime timestamp;

    // Mongo permite campos opcionales - los que no aplican simplemente no van
    private Map<String, Object> metadatos;
}
```



Un Documento es la manera en la que nombramos una entidad de software que representa una estructura no relacional



Usamos la anotacion de ejemplo EventoRestauranteDocument en codigo para identificar el tipo de objeto



Un Document tambien puede utilizar las anotaciones de lombok asi nos liberamos de realizar manualmente ciertos metodos de clase

CREA LOS DOCUMENTOS Y SUS RELACIONES EN CODIGO



Recuerda Utilizar las anotaciones correctas –
@Document – el nombre que le des a las propiedades
del objeto es el mismo que tendra en su BD

COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

5

Creamos nuestro Repository y lo extendemos de la Interface que nos ofrece Mongo

```
@Repository
public interface EventoRestauranteRepository
    extends MongoRepository<EventoRestauranteDocument, String> {

    List<EventoRestauranteDocument> findByEntidadTipoAndEntidadId(
        String entidadTipo, Long entidadId);

    List<EventoRestauranteDocument> findByTipoAndTimestampBetween(
        String tipo, LocalDateTime desde, LocalDateTime hasta);
}
```



MongoRepository es la interface que nos ofrece Spring data para realizar las operaciones que necesitamos



Es necesario colocar primero el objeto o documento que representa la tabla y luego el tipo del identificador que tiene ese objeto (debe ser el mismo que se establece en el objeto documento)

CREA LOS REPOSITARIOS NECESARIOS



Recuerda es SOLO 1 repositorio por cada documento que creaste



COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?

- 6 Como lo hicimos anteriormente, vamos a inyectar el Repositorio dentro del servicio solo que estaremos gestionando la informacion es en NOSQL
- 7 Aplicamos El mismo patrón de mappers. El dominio no sabe que el almacenamiento es Mongo — el DocumentMapper es quien traduce entre el objeto de negocio y el documento de persistencia.

```
@Mapper(componentModel = "spring")
public interface EventoMapper {

    // Dominio -> Documento (para guardar en Mongo)
    // El id lo asigna Mongo al hacer save()
    @Mapping(target = "id", ignore = true)
    EventoRestauranteDocument toDocument(EventoRestaurante evento);

    // Documento -> Dominio (al leer de Mongo)
    EventoRestaurante toDomain(EventoRestauranteDocument doc);
}
```



COMO IMPLEMENTAR UNA BASE DE DATOS RELACIONAL EN MI API?



Normalmente no necesitamos usar `@EnableMongoRepositories`, Ya que Spring Boot lo detecta automáticamente si los repositorios están en subpaquetes del paquete principal. Solo agrégalo a la clase `@SpringBootApplication` si ves el error No qualifying bean of type `MongoRepository`: `@EnableMongoRepositories(basePackages = "com.tuapp.repository")`. Igual aplica a JPA con `@EnableJpaRepositories`.

```
@SpringBootApplication
@EnableMongoRepositories
public class MdbSpringBootApplication implements CommandLineRunner{

    @Autowired
    ItemRepository groceryItemRepo;

    @Autowired
    CustomItemRepository customRepo;

    List<GroceryItem> itemList = new ArrayList<GroceryItem>();

    public static void main(String[] args) {
        SpringApplication.run(MdbSpringBootApplication.class, args);
    }
}
```



COMO PUEDEN VER EN LAS IMPLEMENTACIONES

la Lógica no cambia — solo cambia de dónde vienen los datos es decir si vienen de su Alacena o de su Nevera

```
// S08: datos en memoria
public List<Plato> obtenerPorCategoria(String categoria) {
    return platos.values().stream()
        .filter(p -> p.getCategoria().equalsIgnoreCase(categoria))
        .toList();
}
```



```
// S09: datos del Repository — solo cambia la fuente y se agrega .map()
public List<Plato> obtenerPorCategoria(String categoria) {
    return platoRepository.findByCategoriaIgnoreCase(categoria).stream()
        .map(entityMapper::toDomain) // ← única diferencia
        .toList();

    // O mejor aún: dejar que la BD filtre y convertir solo el resultado
}
```



DOSW COMPANY

COMO GUARDAR IMAGENES EN BD?

IMAGENES EN DB - RELACIONAL

- Se utiliza un tipo binario: BLOB (MySQL, oracle) o BYTEA (PostgreSQL)
- Pueden guardarla como string siempre y cuando sea una URL que exista en internet
 - La gran desventaja es que si esa imagen se elimina del servidor remoto se rompe el link y toca modificarlo

```
@Entity
public class Product {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @Lob
    @Column(name = "image", columnDefinition = "BLOB")
    private byte[] image;
}
```

```
@PostMapping("/upload")
public ResponseEntity<?> upload(@RequestParam MultipartFile file) throws IOException {
    Product p = new Product();
    p.setName("Test");
    p.setImage(file.getBytes());

    repository.save(p);
    return ResponseEntity.ok().build();
}
```

```
@Entity
public class Product {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    private String imageUrl;
}
```

```
@PostMapping("/upload")
public ResponseEntity<?> upload(@RequestParam MultipartFile file) throws IOException {

    String path = "/images/" + file.getOriginalFilename();
    Files.copy(file.getInputStream(), Paths.get(path));

    Product p = new Product();
    p.setImageUrl(path);

    repository.save(p);

    return ResponseEntity.ok().build();
}
```

IMAGENES EN DB - NO RELACIONAL

- Se puede guardar como un tipo binario
- Se puede guardar utilizando GridFS que es un sistema interno para archivos grandes
- Se puede guardar como URL

```
{  
  "name": "product",  
  "image": BinData(...)  
}
```

```
@Autowired  
private GridFsTemplate gridFsTemplate;  
  
public void saveFile(MultipartFile file) throws IOException {  
    gridFsTemplate.store(  
        file.getInputStream(),  
        file.getOriginalFilename(),  
        file.getContentType()  
    );  
}
```

```
{  
  "name": "product",  
  "imageUrl": "https://cdn.miapp.com/img1.png"  
}
```

PERO AUN TENEMOS UN PROBLEMA... LA IDEMPOTENCIA !!!

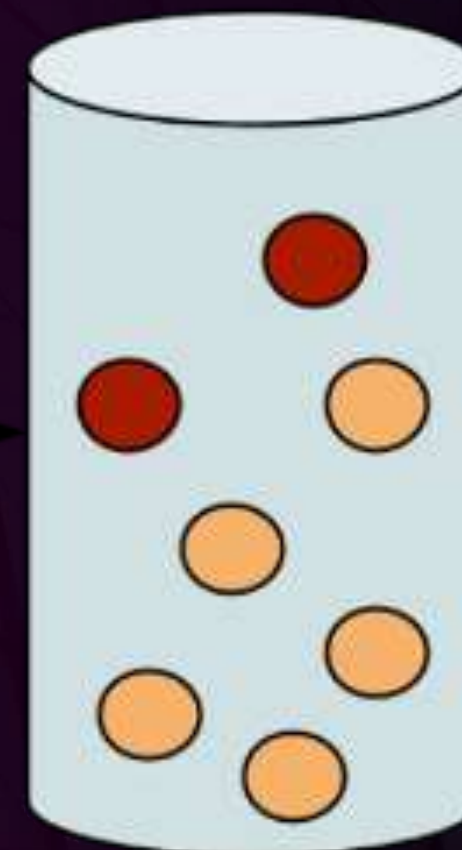
Petición HTTP 2

Verbo NO idempotente

POST /api/v1/pedidos

```
{  
  "descripcion": "leche",  
  "cantidad": 3  
}
```

A
P
I



BBDD

LONG, UUID V4, UUID V7 O STRING — ¿CUÁL USAS?

- El tipo de ID de tu entidad no es un detalle — afecta el rendimiento de los índices, la seguridad de los endpoints, y si puedes generar IDs antes de guardar en la BD. Elige según el caso.





LONG

```
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

// Ejemplo de valor en la BD: 42
// URL generada: GET /api/v1/platos/42
```

- **Cuándo usarlo**

- Tablas con muchas relaciones (FK)
- Datos internos no expuestos en URL
- Performance crítico en JOINS
- Tablas de soporte: categoría, rol, estado

- **Cuándo evitarlo**

- IDs en URLs públicas (usuarios, pedidos)
- Sistemas distribuidos multi-BD
- Cuando el cliente necesita saber el ID antes del INSERT
- APIs expuestas a terceros



UUID V4

```
@Id @GeneratedValue(strategy = GenerationType.UUID)
private UUID id; // Java 17+, Hibernate 6.2+

// Ejemplo: 550e8400-e29b-41d4-a716-446655440000
// URL: GET /api/v1/reservas/550e8400-e29b-41d4-a716-446655440000
```

- **Cuándo usarlo**

- IDs que van en URLs públicas
- No revela cuántos registros hay
- Imposible adivinar IDs ajenos
- Fácil de generar en el cliente antes de enviar

- **Cuidado:**

- Índices más grandes que Long
- Random: fragmenta el índice B-tree (lento en inserts masivos)
- Más bytes en cada FK de tablas relacionadas



UUID V7

```
// Spring Boot 3.2+ / Java 21+ con Hibernate 6.4+
@Id @UuidGenerator(style = UuidGenerator.Style.TIME)
private UUID id;

// O con libreria uuid-creator:
@PrePersist
void generarId() {
    if (id == null) id = UuidCreator.getTimeOrderedEpoch(); // v7
}

// Ejemplo: 018e8f3a-1b2c-7xxx-xxxx-xxxxxxxxxxxx
// Los primeros 48 bits son timestamp — ordenado cronológicamente
```

- **Porque usarlo?**

- UUID: no revela secuencia ni cantidad
- Ordenado por tiempo: no fragmenta el índice
- INSERT tan eficiente como Long en secuencia
- Compatible con sistemas distribuidos
- Se puede generar antes del INSERT

- **Contexto necesario:**

- Requiere Hibernate 6.4+ o librería externa
- El timestamp está en los bits — visible si se decodifica
- Overkill para proyectos pequeños

ID AUTOGENERADO - MONGO

```
@Id
private String id; // Mongo genera ObjectId automáticamente

// Ejemplo: 64f3a1b2c3d4e5f6a7b8c9d0
// 12 bytes: 4 timestamp + 5 random + 3 contador incremental
// Ya es ordenado por tiempo — como UUID v7 para Mongo
```

- **Porque usarlo?**

- Mongo genera ObjectId automáticamente al insertar
- Incluye timestamp — sin fragmentar índice
- Se puede usar UUID si necesitas consistencia cross-BD
- Más corto que UUID (24 hex chars vs 36)

- **Contexto necesario:**

- Si el mismo ID debe existir en Postgres y Mongo
- Usar `@Field("_id")` con tipo UUID
- Mongo lo guarda como Binary subtype 4

COMO ELEGIR EL CORRECTO?

- El gerente quiere enviarle al cliente un link de confirmación de su reserva por WhatsApp. ¿Qué tipo de ID le conviene a la tabla de reservas? **UUID V4**



- El sistema necesita registrar cada cambio de estado de cada pedido como un evento. Se esperan miles de inserts por hora. ¿Qué ID usar para la colección de eventos en MongoDB?

**GENERADO POR
MONGO**



COMO ELEGIR EL CORRECTO?

- La tabla de categorías de platos (Entradas, Principales, Postres, Bebidas) raramente cambia, no se expone en URLs públicas y tiene relaciones con muchas otras tablas.

¿Qué ID usas? **LONG SECUENCIAL**

- El restaurante recibe 500 pedidos por hora. Cada pedido tiene un ID que va en la URL de seguimiento del cliente. Quieres que el índice no se fragmente. ¿Qué ID usas?

UUID V7



TAREA

- Revisen cual tipo de persistencia es mejor para su restaurante – Relacional, No relacional o Hibrida
- La siguiente clase tienen que tener su API completa con persistencia implementada
- INVESTIGUEN LO SIGUIENTE PARA CADA RESTAURANTE:
 - Ya saben quien es el usuario, pero ahora tienen que definir que puede hacer
 - Listen los tipos de Roles que puede tener su restaurante, es decir quien puede crear y eliminar platos? quien puede ver el menu? quien puede tomar pedidos? quien puede cambiarle el estado a un pedido

NOTA – Se pueden guiar del diagrama de contexto – si no lo tienen actualizado es momento

- Para cada rol generen una tabla en excel donde este: ROL, que funcionalidades puede hacer y cuales no Puede hacer





DOSW COMPANY



NOS VEMOS EN
LA SESION DE
AVANCE DEL
PROYECTO